Recursion

It is an algorithmic paradigm used to solve complex problems by breaking them into smaller sub-problems.

Fibonacci Series

0 1 1 2 3 5

1st term	= 0
2nd term	= 1

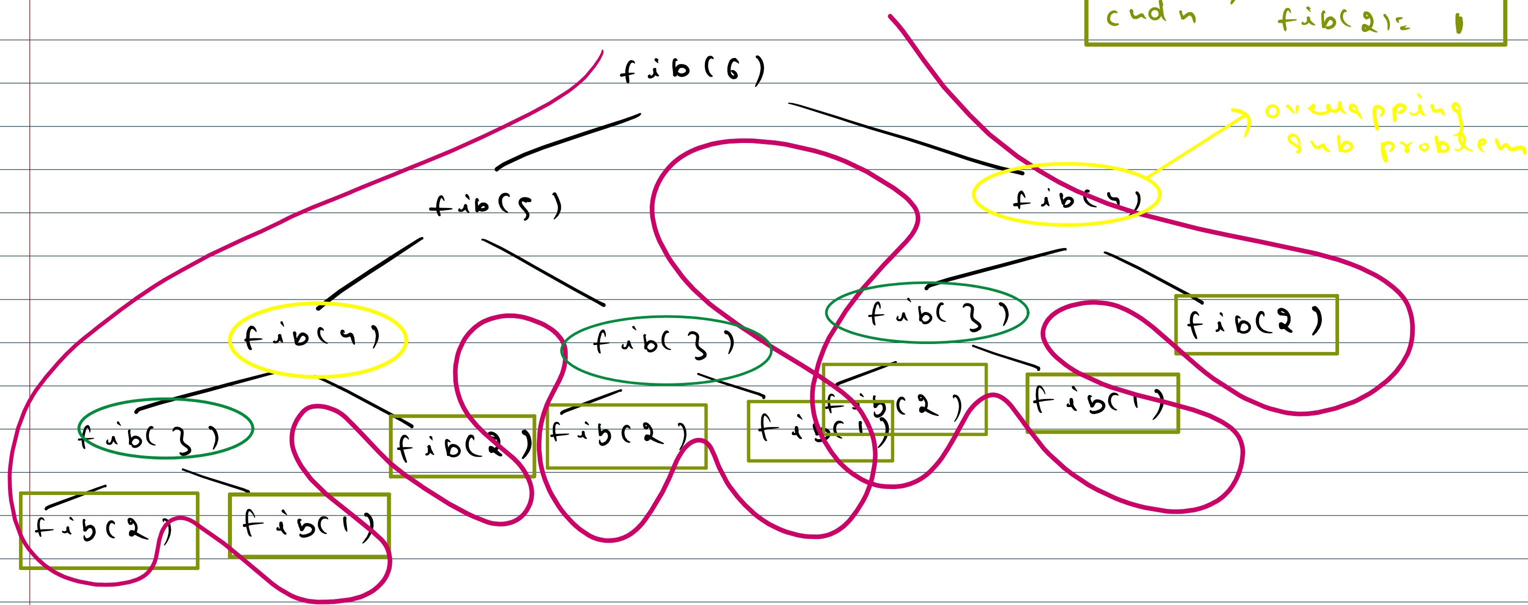
Input => n = 7 => 7 th term => 8
output => nth term of the series

fib(n) = fib(n-1) + fib(n-2)

↓ problem — ↓ sub-problem (1) — ↓ sub-problem (2)

} → optimal sub structure

$\boxed{\text{Base cndn} \Rightarrow fib(1) = 0,\ fib(2) = 1}$



Memorised Recursion (Dynamic Programming)

It is an algorithmic paradigm used to solve complex problems by breaking them into smaller sub-problems, solving each subproblem once, and storing (memorising) them to avoid recomputation.

Key properties of DP

1) overlapping subproblems
2) optimal substructure

Lookup table

to store solved subproblems result, need not to recompute.

Approaches
- → Top-down (memorization) (recursion + storage)
- → Bottom-up (iterative storage)

Knapsack
- → fractional → Greedy
- → 0/1 → DP
- → unbounded → DP

Input :-

wt[] =	1	3	4	5	(KG)
val[] =	1	4	5	7	(INR)

W = 7 kg

O/P → max profit

choices, optimal → DP